

News-Hash: Ein nachvollziehbares Nachrichtenarchiv mit Hash-Ketten und öffentlicher Zeitverankerung

Technischer Projektbericht zu Entwurf, Integritätsmodell und Betrieb eines lokalen Nachrichtenarchivs

News-Hash-Projekt · Version 0.1.49 · 17. August 2026

Zusammenfassung. Nachrichteninhalte sind flüchtig: Artikel werden nachträglich verändert, gekürzt, verschoben oder entfernt. Dieser Beitrag beschreibt News-Hash, ein lokales Nachrichtenarchiv, das konfigurierte JSON- und XML-Quellen regelmäßig abrufen und neue Meldungen in append-only Speicherstrukturen ablegt. Die interne Nachvollziehbarkeit entsteht durch getrennte Hash-Ketten in JSONL und SQLite. Tägliche Manifeste verzeichnen die Ketten und werden optional mit OpenTimestamps öffentlich zeitverankert. Der Beitrag erläutert Architektur, Datenfluss, Hash-Berechnung, quellspezifische Codecs und die Grenzen des Verfahrens. News-Hash beweist nicht die journalistische Wahrheit einer Meldung, sondern die Integrität und zeitliche Existenz der archivierten Fassung.

Schlüsselwörter: Nachrichtenarchiv, Hash-Kette, OpenTimestamps, digitale Langzeitarchivierung, Datenintegrität

1. Problemstellung und Zielsetzung

Die öffentliche Nachrichtenkommunikation ist auf Aktualität und Veränderbarkeit ausgelegt. Genau diese Eigenschaften erschweren eine spätere Rekonstruktion: Ein Link kann auf eine redigierte Fassung zeigen oder vollständig ins Leere laufen. Für wissenschaftliche, journalistische und gesellschaftliche Einordnung ist jedoch relevant, welche Information zu einem bestimmten Zeitpunkt vorlag.

News-Hash verfolgt deshalb ein begrenztes, technisch überprüfbares Ziel. Die Anwendung soll konfigurierte Nachrichtenquellen regelmäßig erfassen, neue Datensätze dauerhaft lokal speichern und nachträgliche Veränderungen erkennbar machen. Die Lösung ist kein Wahrheitssystem und keine eigene Blockchain. Sie ist ein Archivierungsdienst mit einer internen Verketzung von Datensätzen und einem optionalen externen Existenznachweis.

2. Systemarchitektur

Die Python-Anwendung besteht aus einem Kommandozeilenprogramm, einer Import- und Normalisierungsschicht, austauschbaren Quellen-Codecs, zwei Speichern und einer eingebetteten HTTP-WebUI. Quellen werden in einer TOML-Datei mit URL, Speichername, Codec und individuellem Abrufintervall definiert. Der Daemon verarbeitet die Quellen unabhängig voneinander; ein Fehler bei einer Quelle soll die übrigen Quellen nicht anhalten.

JSONL und SQLite werden parallel geführt. JSONL ist als einfaches, zeilenorientiertes Austausch- und Archivformat lesbar. SQLite unterstützt schnelle Abfragen, Filterung und Detailsichten im Dashboard. Beide Formate verwenden dasselbe fachliche Hash-Material, führen aber jeweils eine eigene Kette. Diese Trennung verhindert, dass ein unvollständiger Schreibvorgang in einem Format automatisch die Integrität des anderen aufhebt.

Große Dateien werden in nummerierte Shards aufgeteilt. Die Laufzeit prüft das Schema von SQLite beim Öffnen und benennt inkompatible Alt-Tabellen um, statt sie stillschweigend zu überschreiben. Dadurch bleiben historische Daten bei Schemaänderungen erhalten.

3. Datenfluss und Normalisierung

Ein Quellenlauf beginnt mit dem HTTP-Abwurf eines JSON- oder XML-Feeds. Das Feedformat wird in eine interne Darstellung überführt. Der ausgewählte Codec normalisiert Titel, Inhalt, URL, Autor, Veröffentlichungszeit und Bilder. Bereits bekannte `source_id`-Werte werden vor teuren Verarbeitungsschritten übersprungen. Neue Datensätze werden anschließend in beide Speicher geschrieben.

Die Codec-Schnittstelle kapselt Besonderheiten einzelner Anbieter. RSSv0 verarbeitet allgemeine Feeds und lädt Bilder aus HTML-Inhalten. TAZv0 ruft ergänzend den vollständigen Artikel über den Feed-Link ab. SCREENv0 öffnet den ersten Link mit Chromium, entfernt bekannte Consent- und Cookie-Banner und speichert einen visuellen PNG-Schnappschuss. So bleibt die allgemeine Importlogik unabhängig von host- oder quellspezifischen Regeln.

4. Integritätsmodell

Für jeden Datensatz wird ein kanonisches JSON-Objekt gebildet. Seine Schlüssel werden sortiert, zusätzliche Leerzeichen vermieden und die UTF-8-Darstellung mit SHA-256 gehasht. Das Material von RSSv0 umfasst unter anderem Quelle, Titel, Inhalt, Veröffentlichungszeit, Codec, Bilder und den Hash des Vorgängers. Der Abrufzeitpunkt wird gespeichert, aber nicht gehasht, damit ein späterer Abruf derselben veröffentlichten Fassung keinen neuen Inhalts-Hash erzeugt.

[illegible]

Ein Datensatz ist gültig, wenn sein gespeicherter `previous_hash` auf den erwarteten Vorgänger zeigt und sein `hash` dem neu berechneten Wert entspricht. Eine nachträgliche Änderung am Inhalt verändert den Hash des betroffenen Datensatzes; wegen der Verkettung werden außerdem alle nachfolgenden Abweichungen sichtbar. Das Modell schützt damit vor unbemerktem Umschreiben der lokalen Archivspur, nicht vor einer falschen oder manipulierten Quelle vor dem Abruf.

5. Öffentliche Zeitverankerung

Einmal täglich werden die jeweils letzten Hashes der JSONL- und SQLite-Kette in einem Manifest zusammengefasst. Das Manifest enthält keine Nachrichtentexte. Mit `ots stamp` kann es an OpenTimestamps übergeben werden. Die daraus entstehende `.ots`-Datei verbindet den Hash mit einem öffentlichen Zeitznachweis und letztlich mit der Bitcoin-Zeitachse.

Zusätzlich können Manifest und Proof-Datei in einem GitHub-Repository versioniert veröffentlicht werden. OpenTimestamps liefert damit die externe zeitliche Einordnung, während GitHub die Nachweisdateien auffindbar und ihre Änderungsgeschichte sichtbar hält. Unveränderte Dateien werden nicht erneut hochgeladen.

6. Anwendung und Betrieb

Die WebUI zeigt Quellenkarten, Kennzahlen, Anchor-Status, Meldungslisten und gespeicherte Detaildaten. Filter und Pagination unterstützen die Untersuchung großer Archive. Zeitstempel werden intern in UTC verarbeitet und im Browser lokal dargestellt. Prometheus-Metriken stellen Quellen-, Datensatz-, Bild- und Fehlerzähler bereit; ein optionaler Heartbeat signalisiert zusätzlich, dass der Daemon läuft.

Die WebUI besitzt bewusst keine Benutzerverwaltung und keine Authentifizierung. Sie ist daher für lokale oder anderweitig geschützte Netze gedacht und darf nicht ungeschützt in ein öffentliches Netz exponiert werden. Die Betriebsüberwachung ersetzt außerdem keine Integritätsprüfung: Ein laufender Prozess kann korrekte Daten ebenso wenig garantieren wie ein Heartbeat.

7. Diskussion und Grenzen

Der wichtigste Vorteil des Entwurfs ist die klare Trennung von Archiv, Integritätsmodell und öffentlichem Nachweis. Lokale Inhalte bleiben unter Kontrolle des Betreibers; die Blockchain muss keine Nachrichten speichern. Die parallelen Speicher erhöhen die Nutzbarkeit und erlauben eine partielle Wiederherstellung. Die Codec-Struktur erleichtert die Erweiterung auf Quellen mit unterschiedlichen technischen Anforderungen.

Dem stehen Grenzen gegenüber. Die Hash-Kette belegt nicht, dass eine Quelle korrekt berichtet, und sie kann nicht beweisen, dass vor dem Abruf keine Manipulation stattfand. Ein lokaler Betreiber kann Daten zurückhalten, fehlerhaft konfigurieren oder den Abruf ausfallen lassen. Auch die öffentliche Zeitverankerung bestätigt primär die Existenz eines Hashes, nicht den Inhalt selbst. Vollständige Unabhängigkeit erfordert daher zusätzlich Vertrauen in Quellen, Abrufzeitpunkt, Konfiguration und Veröffentlichungsprozess.

8. Fazit

News-Hash zeigt, wie sich ein pragmatisches Nachrichtenarchiv mit Standardformaten, SHA-256 und einem externen Zeitznachweis entwerfen lässt. Die Architektur bleibt lokal betreibbar, erweitert aber die interne Nachvollziehbarkeit um einen öffentlich prüfbaren Anker. Damit entsteht keine objektive Wahrheit über Nachrichten, wohl aber eine belastbare technische Spur dessen, was archiviert wurde und wann diese Spur spätestens existierte.

Literatur und Projektdokumentation

[1] News-Hash-Projekt: *Architektur*, `project-docu/architecture.md`, 2026.

[2] News-Hash-Projekt: *Anforderungen*, `project-docu/requirements.md`, 2026.

[3] OpenTimestamps: *OpenTimestamps* *Documentation*,
<https://opentimestamps.org/>.

Dieser Artikel beschreibt den Stand des Projekts zum 17. August 2026.